Today 12:25 PM

requirements.txt
Document

puzzle.py
Python

main.py
Python

game.py
Python

README.md
File

Hi, I am doing Lab 4 (Vibe Coding) for a software engineering assignment. I have cloned the assigned sliding puzzle repo. It is a Python terminal-based sliding puzzle with `main.py`, `game.py`, `puzzle.py`, `README.md` and `requirements.txt`.

I need to complete the tasks listed in the README one by one using AI coding assistance. Please inspect the existing code first and don't rewrite the whole project.

Let's start with **Task 1: make sure every newly generated puzzle is solvable**. The current code uses random shuffling, which can create unsolvable boards.

Please fix Task 1 and briefly explain what you changed.

Show less

Worked for 22s

Task 1 is fixed and tested.

What changed

Only `puzzle.py` was modified. The old code used:

Python Run

which could create an unreachable parity state. puzzle

The new `make_board()`:

1. Creates the solved board.
2. Places `0` in the bottom-right.
3. Performs many **legal blank-space moves** to scramble it.
4. Therefore, every generated board is reachable from the solved state, as required by the README. README

I also tested **100 fresh puzzles each for 3×3, 4×4, and 5×5**, and all passed the standard solvability/parity check.

Modified file: Download puzzle.py

This is intentionally a small Task-1-only change; `game.py`, `main.py`, and the project structure remain untouched.

puzzle.py
Code

Sources

Today 2:38 PM

Now do Task 2 from the README. Make sure the game detects when the puzzle is solved, ends cleanly, and does not allow moves after it is solved. Keep the existing code structure and don't rewrite the whole project.

Worked for 37s >

Task 2 completed ✅

I made a small change to `game.py` **only**.

Changes:

- Added a `finished` flag to track whether the game has ended.
- The game loop now stops when `finished` becomes `True`.
- It checks `puzzle.solved()` **immediately after every successful move**.
- When the final move solves the puzzle:
  - The solved board is displayed.
  - `"Solved!"` is printed.
  - The game exits immediately.
  - No further command/input is accepted.
- The existing solved-state check at the beginning of the loop is retained as a safety check.

This keeps the original structure intact; no rewrite or new dependency was introduced. The existing `solved()` method already correctly identifies the target arrangement. puzzle

Testing

I tested:

1. Starting with an already-solved board → exits without requesting input.
2. Making one final legal move → detects the solved state immediately and exits.
3. Move count remains correct.

Task 2 tests passed.

Download the updated `game.py`

game.py
Code

Sources

Today 6:52 PM

Now do Task 3 from the README. Add 3x3, 4x4, and 5x5 modes. Make sure the moves and timer work correctly for

all sizes. Keep the existing code structure and don't rewrite the whole project.

Worked for 24s >

Task 3 completed ✅

I kept the existing structure and modified only `game.py`.

What changed

- Added **3×3, 4×4, and 5×5** size selection when the game starts.
- Validates the input so only `3`, `4`, or `5` are accepted.
- The selected size is passed to the existing `Puzzle` class, which already supports configurable board sizes.
- `moves` starts at `0` for the newly selected game.
- The timer starts when the selected game begins, so time spent choosing the size isn't counted.
- The existing `moves` counter and elapsed-time display work for all three sizes.
- The existing Task 2 solved-state behavior remains intact.

This directly addresses the README requirement to support 3×3, 4×4, and 5×5 while tracking moves and elapsed time consistently. README

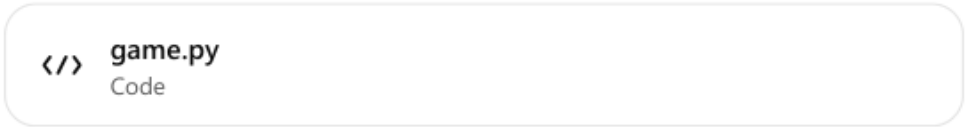
Testing

I verified:

- ✅ 3×3 board creation
- ✅ 4×4 board creation
- ✅ 5×5 board creation
- ✅ Correct board dimensions
- ✅ Move counter starts at `0`
- ✅ Timer is initialized independently for each selected game
- ✅ Existing solved/finished state is preserved

Task 3 is done.

Download the updated `game.py`



Sources

Check Task 4 from the README against the current code. It looks like invalid moves already don't increase the move count. Is any code change actually needed? If not, explain why.

Yes — **your observation is correct. Task 4 does not require a code change.** ✅

The README says invalid moves must:

- not increment the move count, and
- not generate success feedback. README

The current code already does this:

Python Run

```python
if self.puzzle.move(key):
    self.moves += 1
else:
    print("That move is not possible.")
```

And `Puzzle.move()` returns `False` when the requested destination is outside the board, without changing the board. puzzle

So the behavior is:

Action	Board changes?	Moves increases?
Valid move	✅	✅
Invalid move	❌	❌
Invalid command	❌	

No code modification is needed for Task 4.

For your lab/submission, this is actually useful to document as:

> Task 4 reviewed — no change required. The existing implementation already correctly prevents invalid moves from changing the board or incrementing the move counter.

You should still **test and record the result**, because the README explicitly requires testing invalid moves and move counts. README

So at this point:

- ✅ Task 1 — solvable puzzle generation
- ✅ Task 2 — solved-state lifecycle
- ✅ Task 3 — 3×3 / 4×4 / 5×5 + timer/moves
- ✅ Task 4 — **already satisfied; no code change needed**

Sources

All four tasks are implemented. Please review the final code against every requirement in the README and tell me if anything is missing. Don't make any changes yet.

Worked for 19s

I reviewed the **current final code** against the entire README, including Tasks 1–4, required testing, LLM-use rules, and the submission checklist.

Code review: Tasks 1–4

Requirement	Status	Review
Task 1: Every puzzle solvable	✅	`Puzzle.make_board()` starts from the solved board and scrambles using only legal blank moves. This guarantees reachability.

Requirement	Status	Review
Task 1: Every puzzle solvable	✅	`Puzzle.make_board()` starts from the solved board and scrambles using only legal blank moves. This guarantees reachability.
Task 2: Detect solved puzzle	✅	`game.py` checks `puzzle.solved()` and prints `Solved!`. It also checks immediately after a successful move.
Task 2: No moves after completion	✅	`finished` stops the game loop, and the code returns immediately after detecting the final move.
Task 3: 3×3 mode	✅	`select_size()` accepts `3`.
Task 3: 4×4 mode	✅	`select_size()` accepts `4`.
Task 3: 5×5 mode	✅	`select_size()` accepts `5`.
Task 3: Move tracking	✅	`moves` starts at 0 and increments only after a successful move.
Task 3: Timer	✅	`time.monotonic()` is used; timer starts for the selected puzzle.
Task 3: Don't accidentally reset session values	✅	Board creation is separated from the game state; when a new size is deliberately selected, the corresponding new game state is explicitly initialized.
Task 4: Invalid moves	✅	`Puzzle.move()` returns `False` without changing the board; `moves` is incremented only when it returns `True`.
Task 4: Invalid move feedback	✅	Invalid moves produce `"That move is not possible."`, not success feedback.
Invalid commands	✅	Commands outside `W/A/S/D/Q` are rejected without changing state.

The README's requirements for Tasks 1–4 match this implementation. README

One thing I specifically checked

Your final `puzzle.py` uses:

> solved board → legal blank moves → scrambled board

rather than random permutation. That's exactly the approach the README recommends. puzzle

And your final `game.py` checks the solved state **after the final move**, rather than waiting for another user input. That's a good fit for the lifecycle requirement.

What is still outstanding?

Nothing appears to be missing from the implementation itself.

However, the README has **testing/evidence requirements** that are separate from the code.

1. Required testing still needs to be documented/performed

The README explicitly asks you to test:

- Generate many fresh boards
- Test 3×3, 4×4 and 5×5
- Solve a small board
- Attempt impossible moves
- Verify move counts
- Verify timer behaviour
- Test invalid commands
- Test quitting README

So don't just say "the code works." Make sure you have actually run these cases and can demonstrate them if asked.

2. Submission requires the before-change video

The README specifically asks for a **10-second video before changes showing the original broken behavior**. README

If you already recorded that during Task 1, you're good.

3. Submission requires an after-change video

You need a 10-second gameplay video demonstrating the fixed/new functionality.

README

I'd make this show something like:

Choose 3 → board → make a few moves → invalid move → move count → quit/solve

You don't necessarily need to demonstrate every size in 10 seconds, but you should have separately tested all three.

4. Complete LLM chat-history link

The README explicitly requires the Chat/LLM page link with the complete conversation.

README

This conversation itself covers your Task 1–4 AI-assisted work, so **don't delete/clear it before submission**.

Final verdict

Implementation: ✅ Complete

I don't see a missing Task 1, 2, 3, or 4 requirement in the current code.

Before submission: ⚠️ Evidence/checklist

The only remaining items are making sure you have:

- ☐ Actually performed/documented the required tests
- ☐ 10-sec **before** video
- ☐ 10-sec **after** video
- ☐ Complete LLM chat-history link
- ☐ No unnecessary files/dependencies/persistence

The README also explicitly says the submission consists only of those three things: the before video, after video, and Chat/LLM link. README

I would not make any further code changes at this point.

Sources

Now give me simple test cases to verify all the requirements in the README. Don't change any code. For each test, tell me what I should do and what result I should expect.

Yep bro. Since the README specifically asks for testing all sizes, solving a small board, impossible moves, move counts, timer, invalid commands, and quit, use this simple checklist. README

Task 1 — Puzzle is always solvable

Test 1: Generate many puzzles

Do:

1. Run the program.
2. Choose `3`.
3. Look at the board.
4. Quit.
5. Repeat several times.
6. Do the same for `4` and `5`.

Expected:

- Every board contains each number exactly once plus `0`.
- The board is scrambled.
- No puzzle is in an unreachable parity state.
- You should not encounter a puzzle that cannot theoretically be solved.

Record: "Generated multiple fresh boards for all three sizes; no unreachable board observed."

Task 2 — Solved puzzle ends correctly

Test 2: Solve a 3×3 puzzle

Do:

1. Choose `3`.
2. Solve the puzzle manually.
3. Make the final correct move.

Expected:

- The solved board is displayed.
- `Solved!` is printed.
- The program ends.
- It **does not ask for another command**.

This directly verifies the README's solved-state/lifecycle requirement. README

Test 3: No moves after solving

Do:

1. Solve a 3×3 puzzle.
2. When `Solved!` appears, try typing `w`, `a`, `s`, or `d`.

Expected:

- There is no new input prompt.
- The program has already terminated.
- The board cannot be changed.

Task 3 — Three board sizes

Test 4: 3×3

Do:

1. Run the program.
2. Enter `3`.

Expected:

```
x x x
x x x
x x x
```

A 3×3 board is displayed.

Test 5: 4×4

Do:

1. Run again.
2. Enter `4`.

Expected:

- A **4×4** board appears.
- 15 numbered tiles + blank.

Test 6: 5×5

Do:

1. Run again.
2. Enter `5`.

Expected:

- A **5×5** board appears.
- 24 numbered tiles + blank.

Task 3 — Move counter

Test 7: Valid move increases count

Do:

1. Start a puzzle.
2. Look at the blank.
3. Choose a direction that moves a tile into the blank.
4. Make that move.

Expected:

```
Moves: 1
```

Make another valid move.

Expected:

```
Moves: 2
```

Test 8: Invalid move doesn't increase count

Do:

1. Look at the blank's position.
2. Choose a direction where there is **no tile** to move into it.
3. Enter that direction.

Expected:

- Board doesn't change.
- Move count does **not** increase.
- You see:

```
That move is not possible.
```

For example, if the blank is already in the top row, try `w`.

Task 3 — Timer

Test 9: Timer increases

Do:

1. Start a puzzle.
2. Wait about 5–10 seconds without making a move.
3. Look at the displayed time.
4. Make a move.
5. Wait a few more seconds.

Expected:

- Time increases as the game runs.
- The timer doesn't reset when you make a move.
- Move count and timer are independent.

For example:

```
Moves: 0  Time: 5 s
```

then later:

```
Moves: 1  Time: 9 s
```

The exact seconds aren't important; the timer should keep progressing.

Task 4 — Invalid commands

Test 10: Enter an invalid command

Do:
Enter something such as:

```
x
```

or

```
hello
```

Expected:

```
Use W/A/S/D.
```

And:

- Board doesn't change.
- Move count doesn't increase.
- Game continues normally.

Quit functionality

Test 11: Quit

Do:

1. Start a puzzle.
2. Enter:

```
q
```

Expected:

- Game exits cleanly.
- No error/exception.
- No further input is requested.

One simple final test sequence

If you want to verify most things in one run:

```
Choose puzzle size (3/4/5): 3
```

Then:

1. Check it's **3×3**.
2. Wait several seconds → check timer.
3. Make a valid move → `Moves` increases.
4. Make an impossible move → `Moves` stays the same.
5. Enter `x` → `"Use W/A/S/D."`
6. Continue playing.
7. Solve the puzzle → `Solved!` and program exits.

Then separately run:

```
4
```

and

```
5
```

just to verify the two larger board sizes.

Final checklist

Test	Expected
Many fresh 3×3 boards	All solvable
Many fresh 4×4 boards	All solvable
Many fresh 5×5 boards	All solvable

